Algorithms and Computability

Lecture 5: Time Complexity

Martin Zimmermann (Aalborg University)

Last Week in Algorithms and Computability

We have seen

- mapping reductions,
- non-computable problems and how to prove them non-computable via reductions, and
- Rice's theorem: everything "interesting" about Turing machines is not computable.
- Consequence: everything "interesting" about programs is not computable.

Agenda

1. **Motivation**
2. Big-O Notation
3. Complexity Theory
4. Time Complexity

Remember the Entscheidungsproblem?

Lecture 2

The "Entscheidungsproblem" (Hilbert and Ackermann, 1928)

> *Is there an algorithm that, given a statement in some logical language (typically predicate logic), answers "Yes" or "No" according to whether the statement is universally valid.*

Remember the Entscheidungsproblem?

Lecture 2

The "Entscheidungsproblem" (Hilbert and Ackermann, 1928)

> *Is there an algorithm that, given a statement in some logical language (typically predicate logic), answers "Yes" or "No" according to whether the statement is universally valid.*

In our language: Is $\{w \mid w \text{ encodes valid statement}\}$ computable?

Lecture 4

The Entscheidungsproblem is not computable.

A Computable Problem

- Presburger Arithmetic (named after Mojżesz Presburger) is a logical system for reasoning about arithmetic using the following axioms:
  - $\forall x.\ \neg(x + 1 = 0)$
  - $\forall x, y.\ (x + 1 = y + 1) \rightarrow x = y$
  - $\forall x.\ x + 0 = x$
  - $\forall x, y.\ x + (y + 1) = (x + y) + 1$
  - $[P(0) \wedge (\forall x.\ P(x) \rightarrow P(x + 1))] \rightarrow \forall x.\ P(x)$ for all Presburger formulas P (the scheme of induction)

A Computable Problem

- Presburger Arithmetic (named after Mojżesz Presburger) is a logical system for reasoning about arithmetic using the following axioms:
  - $\forall x.\ \neg(x + 1 = 0)$
  - $\forall x, y.\ (x + 1 = y + 1) \rightarrow x = y$
  - $\forall x.\ x + 0 = x$
  - $\forall x, y.\ x + (y + 1) = (x + y) + 1$
  - $[P(0) \wedge (\forall x.\ P(x) \rightarrow P(x + 1))] \rightarrow \forall x.\ P(x)$ for all Presburger formulas P (the scheme of induction)
- With these axioms you can prove, e.g.,

$$\forall x \exists y.\ (x = y + y) \vee (x = y + y + 1)$$

A Computable Problem

- Presburger Arithmetic (named after Mojżesz Presburger) is a logical system for reasoning about arithmetic using the following axioms:
  - $\forall x.\ \neg(x + 1 = 0)$
  - $\forall x, y.\ (x + 1 = y + 1) \rightarrow x = y$
  - $\forall x.\ x + 0 = x$
  - $\forall x, y.\ x + (y + 1) = (x + y) + 1$
  - $[P(0) \wedge (\forall x.\ P(x) \rightarrow P(x + 1))] \rightarrow \forall x.\ P(x)$ for all Presburger formulas P (the scheme of induction)
- The provable statements of Presburger Arithmetic form a computable language **[Presburger '29]**.
- But: Any halting DTM for that language has to move its head at least $2^{2^{cn}}$ times on some inputs of length n (for some constant c) **[Fischer, Rabin '74]**.

Doubly-exponential Growth

Consider $2^{2^{cn}}$ for $c = 1$:

Doubly-exponential Growth

Consider $2^{2^{cn}}$ for $c = 1$:

- $2^{2^0} = 2$

Doubly-exponential Growth

Consider $2^{2^{cn}}$ for $c = 1$:

- $2^{2^0} = 2$
- $2^{2^1} = 4$

Doubly-exponential Growth

Consider $2^{2^{cn}}$ for $c = 1$:

- $2^{2^0} = 2$
- $2^{2^1} = 4$
- $2^{2^2} = 16$

Doubly-exponential Growth

Consider $2^{2^{cn}}$ for $c = 1$:

- $2^{2^0} = 2$
- $2^{2^1} = 4$
- $2^{2^2} = 16$
- $2^{2^3} = 256$

Doubly-exponential Growth

Consider $2^{2^{cn}}$ for $c = 1$:

- $2^{2^0} = 2$
- $2^{2^1} = 4$
- $2^{2^2} = 16$
- $2^{2^3} = 256$
- $2^{2^4} = 65536$

Doubly-exponential Growth

Consider $2^{2^{cn}}$ for $c = 1$:

- $2^{2^0} = 2$
- $2^{2^1} = 4$
- $2^{2^2} = 16$
- $2^{2^3} = 256$
- $2^{2^4} = 65536$
- $2^{2^5} = 4294967296$

Doubly-exponential Growth

Consider $2^{2^{cn}}$ for $c = 1$:

- $2^{2^0} = 2$
- $2^{2^1} = 4$
- $2^{2^2} = 16$
- $2^{2^3} = 256$
- $2^{2^4} = 65536$
- $2^{2^5} = 4294967296$
- $2^{2^6} = 18446744073709551616$

Doubly-exponential Growth

Consider $2^{2^{cn}}$ for $c = 1$:

- $2^{2^0} = 2$
- $2^{2^1} = 4$
- $2^{2^2} = 16$
- $2^{2^3} = 256$
- $2^{2^4} = 65536$
- $2^{2^5} = 4294967296$
- $2^{2^6} = 18446744073709551616$
- $2^{2^7} = 340282366920938463463374607431768211456$

Doubly-exponential Growth

Consider $2^{2^{cn}}$ for $c = 1$:

- $2^{2^0} = 2$
- $2^{2^1} = 4$
- $2^{2^2} = 16$
- $2^{2^3} = 256$
- $2^{2^4} = 65536$
- $2^{2^5} = 4294967296$
- $2^{2^6} = 18446744073709551616$
- $2^{2^7} = 340282366920938463463374607431768211456$
- $\cdots$

A Change in Perspective

We studied the following question:

What problems can be solved by a computer?

- The computable problems are those that can be solved by a computer.
- The computably-enumerable problems are those where a "yes"-answer can be verified by a computer.
- Mapping reductions can be used to find relations between problems.

A Change in Perspective

We will study the following question:

What problems can be solved efficiently by a computer?

- The complexity class P contains the problems that can be solved efficiently by a computer.
- The complexity class NP contains the problems where a "yes"-answer can be verified efficiently by a computer.
- Polynomial-time reductions can be used to find relations between problems.

Agenda

1. Motivation
2. **Big-O Notation**
3. Complexity Theory
4. Time Complexity

Reminder

Definition

Let $f, g \colon \mathbb{N} \to \mathbb{R}_{>0}$ be functions. We write $f(n) = \mathcal{O}(g(n))$ if

- there are positive integers c, n_0 such that
- $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

Reminder

Definition

Let $f, g \colon \mathbb{N} \to \mathbb{R}_{>0}$ be functions. We write $f(n) = \mathcal{O}(g(n))$ if

- there are positive integers c, n_0 such that
- $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

Examples

- $2n^2 + 5 = \mathcal{O}(n^2)$ (pick $c = 3$ and $n_0 = 3$)
- $7n^5 + 88n^4 + n^2 + 2013 = \mathcal{O}(n^5)$
- $\log_2(n^7) = \mathcal{O}(\log_2 n)$

Reminder

Definition

Let $f, g \colon \mathbb{N} \to \mathbb{R}_{>0}$ be functions. We write $f(n) = \mathcal{O}(g(n))$ if

- there are positive integers c, n_0 such that
- $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

Examples

- $2n^2 + 5 = \mathcal{O}(n^2)$ (pick $c = 3$ and $n_0 = 3$)
- $7n^5 + 88n^4 + n^2 + 2013 = \mathcal{O}(n^5)$
- $\log_2(n^7) = \mathcal{O}(\log_2 n)$

Reading

CLRS Section 3.1 and 3.2 on Big-O notation.

Why Do We Care about Growth Rates?

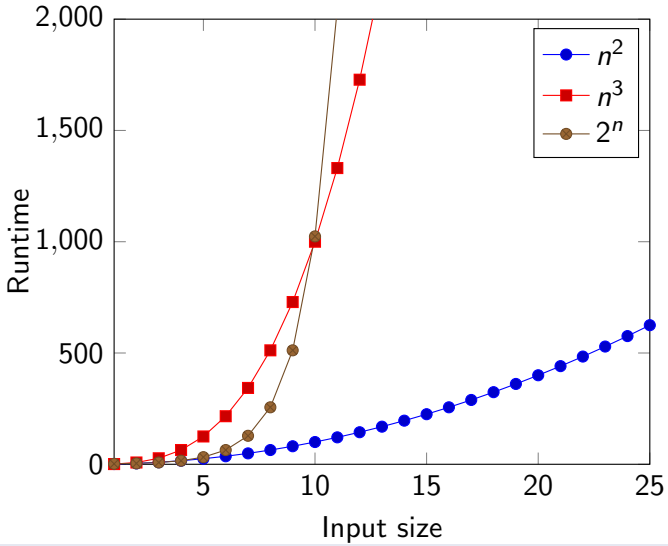
Assume your computer performs 1 billion steps per second. The table shows the CPU time for inputs of size n.

n	$f(n) = n$	$f(n) = n^2$	$f(n) = n^3$	$f(n) = 2^n$
10	0.01 microsec	0.1 microsec	1 microsec	1 microsec
20	0.02 microsec	0.4 microsec	8 microsec	1 millisec
50	0.05 microsec	2.5 microsec	125 microsec	3 days
100	0.1 microsec	10 microsec	1 millisec	$4 \cdot 10^{13}$ years

Why Do We Care about Growth Rates?

Assume your computer performs 1 billion steps per second. The table shows the CPU time for inputs of size n.

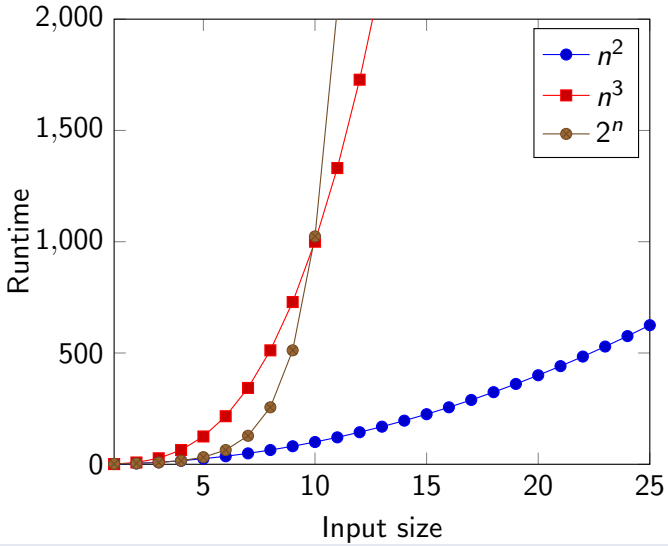
n	$f(n) = n$	$f(n) = n^2$	$f(n) = n^3$	$f(n) = 2^n$
10	0.01 microsec	0.1 microsec	1 microsec	1 microsec
20	0.02 microsec	0.4 microsec	8 microsec	1 millisec
50	0.05 microsec	2.5 microsec	125 microsec	3 days
100	0.1 microsec	10 microsec	1 millisec	$4 \cdot 10^{13}$ years

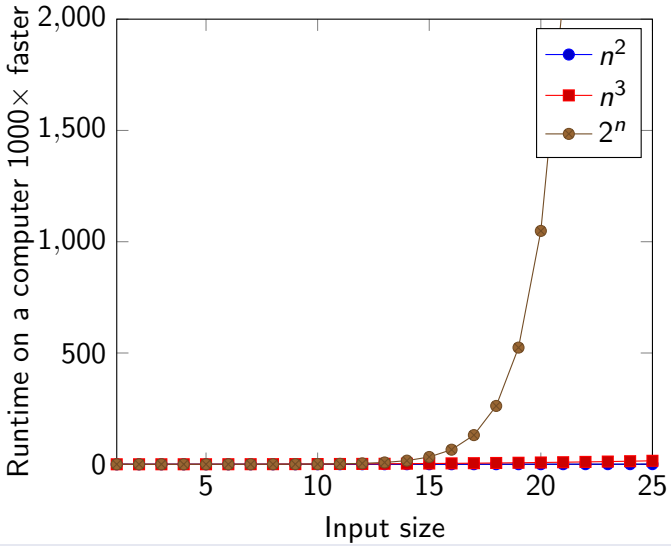


Why Do We Care about Growth Rates?

Assume your computer performs 1 billion steps per second. The table shows the CPU time for inputs of size n.

n	$f(n) = n$	$f(n) = n^2$	$f(n) = n^3$	$f(n) = 2^n$
10	0.01 microsec	0.1 microsec	1 microsec	1 microsec
20	0.02 microsec	0.4 microsec	8 microsec	1 millisec
50	0.05 microsec	2.5 microsec	125 microsec	3 days
100	0.1 microsec	10 microsec	1 millisec	$4 \cdot 10^{13}$ years





Agenda

1. Motivation
2. Big-O Notation
3. **Complexity Theory**
4. Time Complexity

Intuition

Consider the following halting DTM accepting $\{0^m 1^m \mid m \geq 1\}$:

On input w:

1. Pass once over tape content and reject if it is not of the form 0^+1^+.
2. Pass once over the tape content and replace first 0 by X and first 1 by X (if not possible reject).
3. Pass once over the tape content. If there is still a 0 but no 1 or no 0 but still a 1 then reject. If no 0 and no 1 are left accept.
4. Goto 2.

How many steps does the Turing machine take on an input of length n at most?

Intuition

Consider the following halting DTM accepting $\{0^m 1^m \mid m \geq 1\}$:

On input w:

1. Pass once over tape content and reject if it is not of the form 0^+1^+. n steps
2. Pass once over the tape content and replace first 0 by X and first 1 by X (if not possible reject).
3. Pass once over the tape content. If there is still a 0 but no 1 or no 0 but still a 1 then reject. If no 0 and no 1 are left accept.
4. Goto 2.

How many steps does the Turing machine take on an input of length n at most?

Intuition

Consider the following halting DTM accepting $\{0^m 1^m \mid m \geq 1\}$:

On input w:

1. Pass once over tape content and reject if it is not of the form 0^+1^+. n steps
2. Pass once over the tape content and replace first 0 by X and first 1 by X (if not possible reject). $2n$ steps
3. Pass once over the tape content. If there is still a 0 but no 1 or no 0 but still a 1 then reject. If no 0 and no 1 are left accept.
4. Goto 2.

How many steps does the Turing machine take on an input of length n at most?

Intuition

Consider the following halting DTM accepting $\{0^m 1^m \mid m \geq 1\}$:

On input w:

1. Pass once over tape content and reject if it is not of the form 0^+1^+. n steps
2. Pass once over the tape content and replace first 0 by X and first 1 by X (if not possible reject). $2n$ steps
3. Pass once over the tape content. If there is still a 0 but no 1 or no 0 but still a 1 then reject. If no 0 and no 1 are left accept. $2n$ steps
4. Goto 2.

How many steps does the Turing machine take on an input of length n at most?

Intuition

Consider the following halting DTM accepting $\{0^m 1^m \mid m \geq 1\}$:

On input w:

1. Pass once over tape content and reject if it is not of the form 0^+1^+. n steps
2. Pass once over the tape content and replace first 0 by X and first 1 by X (if not possible reject). $2n$ steps
3. Pass once over the tape content. If there is still a 0 but no 1 or no 0 but still a 1 then reject. If no 0 and no 1 are left accept. $2n$ steps
4. Goto 2. 1 step, executed at most $\frac{n}{2}$ times.

How many steps does the Turing machine take on an input of length n at most?

Intuition

Consider the following halting DTM accepting $\{0^m 1^m \mid m \geq 1\}$:

On input w:

1. Pass once over tape content and reject if it is not of the form 0^+1^+. n steps
2. Pass once over the tape content and replace first 0 by X and first 1 by X (if not possible reject). $2n$ steps
3. Pass once over the tape content. If there is still a 0 but no 1 or no 0 but still a 1 then reject. If no 0 and no 1 are left accept. $2n$ steps
4. Goto 2. 1 step, executed at most $\frac{n}{2}$ times.

How many steps does the Turing machine take on an input of length n at most? $\mathcal{O}(n^2)$ steps overall.

Complexity of Algorithms

On the previous slide, we have analyzed the (worst-case) time-complexity of one halting DTM (algorithm) for $\{0^m 1^m \mid m \geq 1\}$.. but there are many different DTM's!

Complexity of Algorithms

On the previous slide, we have analyzed the (worst-case) time-complexity of one halting DTM (algorithm) for $\{0^m 1^m \mid m \geq 1\}$.. but there are many different DTM's!

Example

Consider the sorting problem:

1. Insertion sort requires n^2 comparisons in the worst case.
2. Mergesort requires $n \log_2 n$ comparisons in the worst case.
3. Every comparison-based sorting algorithm requires $n \log_2 n$ comparisons in the worst case.

Complexity of Algorithms

On the previous slide, we have analyzed the (worst-case) time-complexity of one halting DTM (algorithm) for $\{0^m 1^m \mid m \geq 1\}$.. but there are many different DTM's!

Example

Consider the sorting problem:

1. Insertion sort requires n^2 comparisons in the worst case.
2. Mergesort requires $n \log_2 n$ comparisons in the worst case.
3. Every comparison-based sorting algorithm requires $n \log_2 n$ comparisons in the worst case.

Note

The first two statements are about specific algorithms, the third one is about all (comparison-based) algorithms.

Complexity Theory

Focus of algorithm complexity:

- Study concrete algorithms and their complexity.

Focus of complexity theory:

- Study problems (i.e., languages) instead of algorithms.
- Goal: classify problems according to how easy/hard they are to solve.

Complexity Theory

Focus of algorithm complexity:

- Study concrete algorithms and their complexity.

Focus of complexity theory:

- Study problems (i.e., languages) instead of algorithms.
- Goal: classify problems according to how easy/hard they are to solve.

From now on:

- Today, we only consider halting DTM's, i.e., only deterministic Turing machines that halt on every input.
- We analyze the (worst-case) time-complexity of DTM's. Why Turing machines?

Extended Church–Turing thesis

Everything that can be efficiently computed can be computed efficiently by a probabilistic (!) Turing machine.

Extended Church–Turing thesis

Everything that can be efficiently computed can be computed efficiently by a probabilistic (!) Turing machine.

- Under (unproven, but widely assumed) complexity-theoretic assumptions: the word "probabilistic" can be dropped.

Extended Church–Turing thesis

Everything that can be efficiently computed can be computed efficiently by a probabilistic (!) Turing machine.

- Under (unproven, but widely assumed) complexity-theoretic assumptions: the word "probabilistic" can be dropped.
- Under (unproven, but widely assumed) complexity-theoretic assumptions: The extended Church-Turing thesis is false! Efficient quantum Turing machines cannot be simulated efficiently by (even probabilistic) Turing machines.

Extended Church–Turing thesis

Everything that can be efficiently computed can be computed efficiently by a probabilistic (!) Turing machine.

- Under (unproven, but widely assumed) complexity-theoretic assumptions: the word "probabilistic" can be dropped.
- Under (unproven, but widely assumed) complexity-theoretic assumptions: The extended Church-Turing thesis is false! Efficient quantum Turing machines cannot be simulated efficiently by (even probabilistic) Turing machines.

Nevertheless, Turing machines are a robust model for computation and can efficiently simulate almost all other models of computation.

Agenda

1. Motivation

2. Big-O Notation

3. Complexity Theory

4. **Time Complexity**

Runtime of a Deterministic Turing Machine

Definition

Let M be a halting DTM.

- Let $time_M(w)$ denote the number of configurations in the unique run of M on input w.
- Let $T\colon \mathbb{N} \to \mathbb{R}_{>0}$. We say that M runs within time T if $time_M(w) \leq T(|w|)$ for all inputs w.

Runtime of a Deterministic Turing Machine

Definition

Let M be a halting DTM.

- Let $time_M(w)$ denote the number of configurations in the unique run of M on input w.
- Let $T\colon \mathbb{N} \to \mathbb{R}_{>0}$. We say that M runs within time T if $time_M(w) \leq T(|w|)$ for all inputs w.

We are interested in classifying problems (languages) according to their asymptotic time-complexity.

Definition

Let $T\colon \mathbb{N} \to \mathbb{R}_{>0}$ be a function. The complexity class $\textsc{Time}(T)$ is defined as

$$\textsc{Time}(T) = \{L(M) \mid M \text{ is a halting DTM that runs within time } \mathcal{O}(T)\}.$$

Time Hierarchy

A consequence of a more general theorem:

$$\textsc{Time}(n) \subsetneq \textsc{Time}(n^2) \subsetneq \textsc{Time}(n^3) \subsetneq \textsc{Time}(n^4) \subsetneq \cdots \subsetneq \textsc{Time}(2^n)$$

Intuitively: more time allows you to compute more languages.

Proof: via diagonalization construct a language that is different from every language in $\textsc{Time}(n^k)$, but in $\textsc{Time}(n^{k+1})$. Note that these languages are all artificial.

Robustness

For one-tape halting DTM's:

- $\{0^m 1^{m'} \mid m, m' \geq 1\} \in \textsc{Time}(n)$
- $\{0^m 1^m \mid m \geq 1\} \in \textsc{Time}(n^2)$
- $\{w\#w \mid w \in \mathbb{B}^*\} \in \textsc{Time}(n^2)$

Robustness

For one-tape halting DTM's:

- $\{0^m 1^{m'} \mid m, m' \geq 1\} \in \textsc{Time}(n)$
- $\{0^m 1^m \mid m \geq 1\} \in \textsc{Time}(n^2)$
- $\{w\#w \mid w \in \mathbb{B}^*\} \in \textsc{Time}(n^2)$

For two-tape halting DTM's:

- $\{0^m 1^{m'} \mid m, m' \geq 1\} \in \textsc{Time}(n)$
- $\{0^m 1^m \mid m \geq 1\} \in \textsc{Time}(n)$
- $\{w\#w \mid w \in \mathbb{B}^*\} \in \textsc{Time}(n)$

Robustness

For one-tape halting DTM's:

- $\{0^m 1^{m'} \mid m, m' \geq 1\} \in \textsc{Time}(n)$
- $\{0^m 1^m \mid m \geq 1\} \in \textsc{Time}(n^2)$
- $\{w\#w \mid w \in \mathbb{B}^*\} \in \textsc{Time}(n^2)$

For two-tape halting DTM's:

- $\{0^m 1^{m'} \mid m, m' \geq 1\} \in \textsc{Time}(n)$
- $\{0^m 1^m \mid m \geq 1\} \in \textsc{Time}(n)$
- $\{w\#w \mid w \in \mathbb{B}^*\} \in \textsc{Time}(n)$

In general:

Theorem

Let $t\colon \mathbb{N} \to \mathbb{R}_{>0}$ be a function such that $t(n) \geq n$. Every k-tape halting DTM with time-complexity $t(n)$ can be simulated by an equivalent one-tape halting DTM with time-complexity $t^2(n)$.

Robustness

For one-tape halting DTM's:

- $\{0^m 1^{m'} \mid m, m' \geq 1\} \in \textsc{Time}(n)$
- $\{0^m 1^m \mid m \geq 1\} \in \textsc{Time}(n^2)$
- $\{w\#w \mid w \in \mathbb{B}^*\} \in \textsc{Time}(n^2)$

For two-tape halting DTM's:

- $\{0^m 1^{m'} \mid m, m' \geq 1\} \in \textsc{Time}(n)$
- $\{0^m 1^m \mid m \geq 1\} \in \textsc{Time}(n)$
- $\{w\#w \mid w \in \mathbb{B}^*\} \in \textsc{Time}(n)$

In general:

Theorem

Let $t\colon \mathbb{N} \to \mathbb{R}_{>0}$ be a function such that $t(n) \geq n$. Every k-tape halting DTM with time-complexity $t(n)$ can be simulated by an equivalent one-tape halting DTM with time-complexity $t^2(n)$.

Proof: Simulation presented in Lecture 2 has the desired properties.

The Complexity Class P

Definition

The complexity class P (polynomial time) is defined as

$$\mathrm{P} = \bigcup_{k \geq 0} \textsc{Time}(n^k)$$

- Robust definition (can use other deterministic (!) models of computation, e.g., multi-tape Turing machines).
- Cobham's thesis: A problem can be efficiently computed if and only if it is in P.

About Cobham's Thesis

A problem with fastest algorithm with running time..

- n^{58}: efficient!
- $n^{\log_2 \log_2 \log_2 \log_2 n}$: not efficient!

About Cobham's Thesis

A problem with fastest algorithm with running time..

- n^{58}: efficient! But 2^{58} is roughly the age of the universe in seconds.
- $n^{\log_2 \log_2 \log_2 \log_2 n}$: not efficient!

About Cobham's Thesis

A problem with fastest algorithm with running time..

- n^{58}: efficient! But 2^{58} is roughly the age of the universe in seconds.
- $n^{\log_2 \log_2 \log_2 \log_2 n}$: not efficient! But it is roughly $3n$ for $n = 1.000.000$ and roughly $60n$ for $n = 1.000.000.000$.

About Cobham's Thesis

A problem with fastest algorithm with running time..

- n^{58}: efficient! But 2^{58} is roughly the age of the universe in seconds.
- $n^{\log_2 \log_2 \log_2 \log_2 n}$: not efficient! But it is roughly $3n$ for $n = 1.000.000$ and roughly $60n$ for $n = 1.000.000.000$.
- In practical applications: Even quadratic running time infeasible for data-intensive problems.

Problems in P

Many problems are in P:

- $\{0^m 1^{m'} \mid m, m' \geq 1\} \in \textsc{Time}(n)$
- $\{0^m 1^m \mid m \geq 1\} \in \textsc{Time}(n^2)$
- $\{w\#w \mid w \in \mathbb{B}^*\} \in \textsc{Time}(n^2)$

Problems in P

Many problems are in P:

- $\{0^m 1^{m'} \mid m, m' \geq 1\} \in \textsc{Time}(n)$
- $\{0^m 1^m \mid m \geq 1\} \in \textsc{Time}(n^2)$
- $\{w\#w \mid w \in \mathbb{B}^*\} \in \textsc{Time}(n^2)$
- graph reachability
- NFA emptiness

Problems in P

Many problems are in P:

- $\{0^m 1^{m'} \mid m, m' \geq 1\} \in \textsc{Time}(n)$
- $\{0^m 1^m \mid m \geq 1\} \in \textsc{Time}(n^2)$
- $\{w\#w \mid w \in \mathbb{B}^*\} \in \textsc{Time}(n^2)$
- graph reachability
- NFA emptiness
- Primality
- Linear Programming
- Word problem for contextfree grammars
- Matchings
- .. and many other problems

Closure Properties

For reference:

Theorem

P *is closed under union, intersection, complementation, concatenation, and iteration.*

Proof: The constructions presented in Lecture 3 and Tutorial 3 yield halting DTM's with polynomial time-complexity, when applied to halting DTM's with polynomial time-complexity.

Conclusion

We have compared algorithmic complexity:

- study of concrete algorithms and precise running times
- difference between $\mathcal{O}(n^2)$ and $\mathcal{O}(n^3)$ is huge
- running times depend on model of computation

and complexity theory:

- study of problems (languages) rather than algorithms
- difference between $\mathcal{O}(n^2)$ and $\mathcal{O}(n^3)$ may just depend on choice of model (but people still try to find optimal (for fixed model) algorithms)
- results should be valid for every model of computation

Conclusion

We have compared algorithmic complexity:

- study of concrete algorithms and precise running times
- difference between $\mathcal{O}(n^2)$ and $\mathcal{O}(n^3)$ is huge
- running times depend on model of computation

and complexity theory:

- study of problems (languages) rather than algorithms
- difference between $\mathcal{O}(n^2)$ and $\mathcal{O}(n^3)$ may just depend on choice of model (but people still try to find optimal (for fixed model) algorithms)
- results should be valid for every model of computation

Note

Algorithmic complexity is still important for complexity theory: giving an algorithm with polynomial running time for a problem L implies $L \in \mathrm{P}$.

Reading

Wait till next lecture!